



Nginx as Unix-based Application:

Evolution, Architecture, Performance, Security

Abstract: NGINX (high-performance Unix-based web server whose event-driven, multi-process architecture enables it to handle massive concurrency with low resource usage. Report analyzes architectural foundations (Unix-specific optimizations, performance characteristics, and security mechanisms. Additionally, **current work highlights limitations—such as disk I/O stalls and evolving HTTP/3 support.**

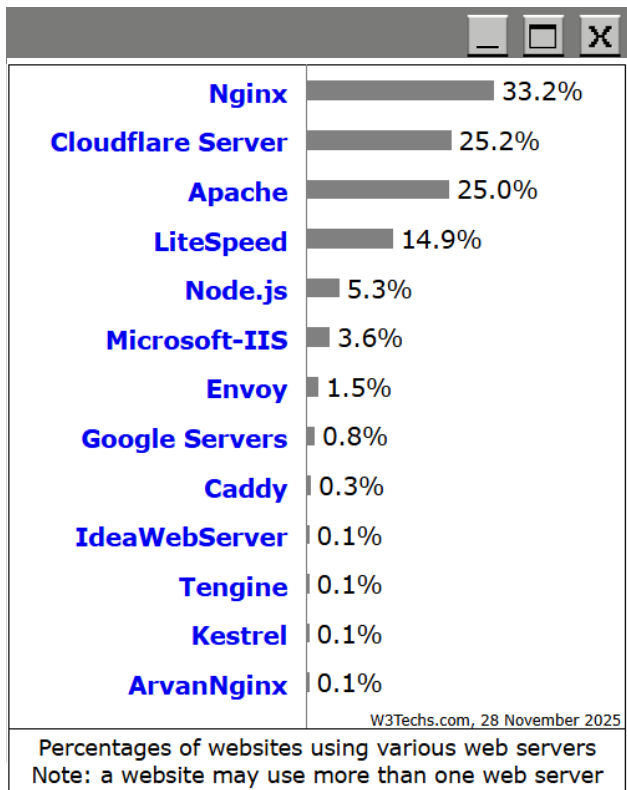


Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.



Introduction. Definition

Nginx – (“engine x”) a free, open-source HTTP web-server written mainly in C language also used as a reverse proxy, load balancer, TCP/UDP proxy server mail proxy server and content cache. Distributed under the two-clause *BSD* (Berkley Unix) license.

Currently nginx is the second most popular open source web server on the Internet.

- High concurrency (clients/browsers) & C10k/C100k in modern complex environment.
- Thread- and process-based servers’ limitations.
- Event-based mechanism.

[01] https://w3techs.com/technologies/overview/web_server

[02] <https://nginx.org/en/>

Evolution of nginx



2002-2004

Igor Sysoev (Rambler, Russia) begins work on NGINX to solve the C10k problem



2011

NGINX Inc. founded, 1.0.0 release in 2011 (gains popularity)



2019

F5 Networks acquires NGINX, Inc. for \$670M (affirming its enterprise value)



2020s

Igor Sysoev **left** Nginx and F5, but core devs are **remain**. F5's is criticized for interference with Nginx's development.

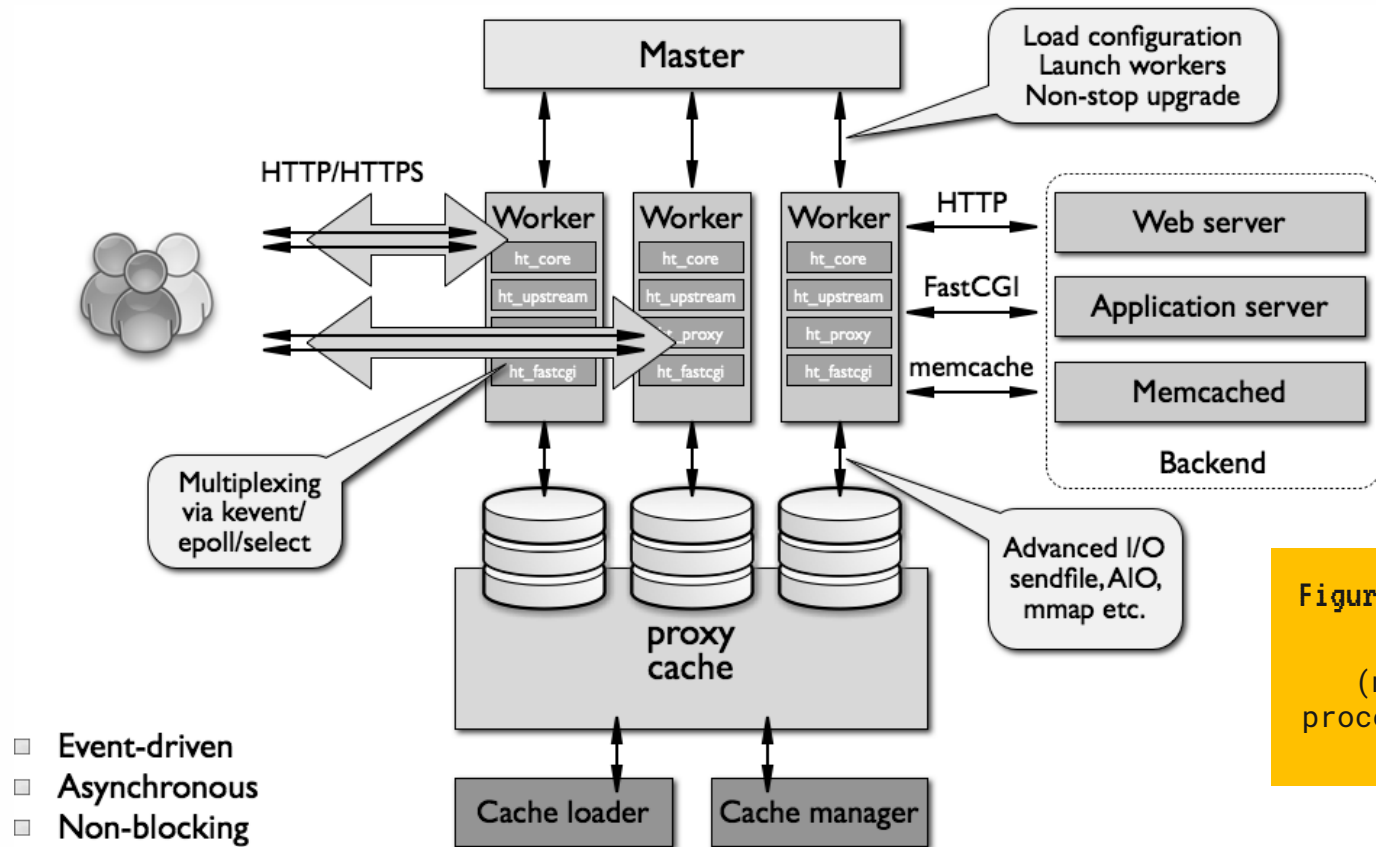


Figure. Diagram of nginx's architecture (master and worker processes using OS event APIs)

Master

- (runs as root) reads config, binds privileged ports, and spawns multiple **worker** processes;
- reloads config or upgrade binaries gracefully by signaling workers (no service interruption).

Worker

- (run as unprivileged user) handle all request processing, each in a single-threaded event loop;
- they share in-memory configuration that can be changed without dropping connections;
- are isolated (one per CPU core recommended) to leverage multicore and avoid locks/threads.

OK

Cancel

Apply



Architecture. Defined.



Each worker uses a single-threaded event loop to handle all connections.

Multiplexing APIs: epoll (Linux), kqueue (BSD), or poll/select on older Unix.

Workflow: on new connection, `accept()` then add socket to event loop; on readiness, read request, process, write reply, close or keep-alive.

Non-blocking I/O everywhere: socket reads/writes are non-blocking; any blocking OS calls are avoided or delegated.

Uses lightweight callbacks and state machines for HTTP parsing (phases), never spawning a thread per request.

C10K+

can handle tens of thousands of clients on modest hardware

Resource efficiency

Because NGINX uses a small memory footprint per connection, CPU and memory usage remain low under high load (unlike Apache)

N->to->M connections/

scales nearly linearly with cores (one worker per core) and avoids thread thrashing

Event-driven vs Thread

Studies show SPED servers (like NGINX) outperform thread-/process-based servers on in-memory workloads

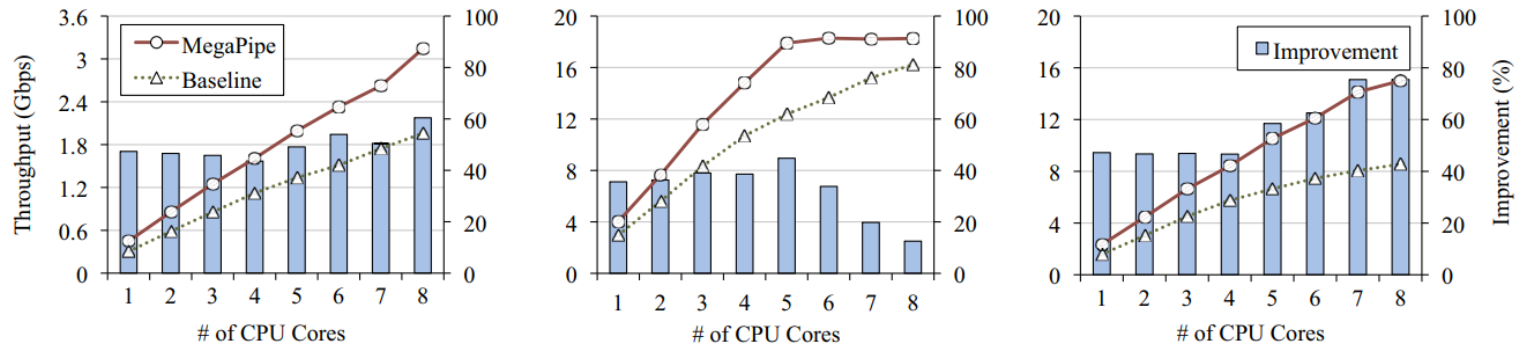
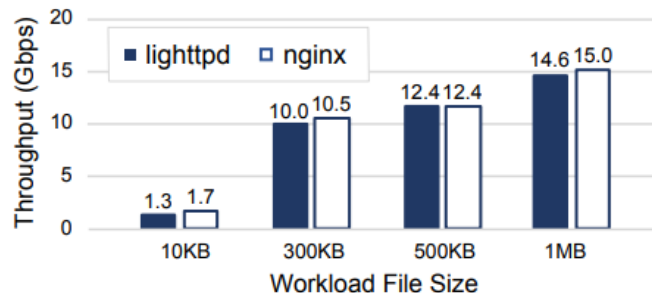
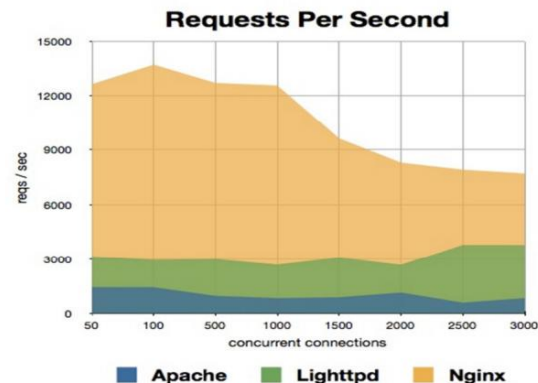
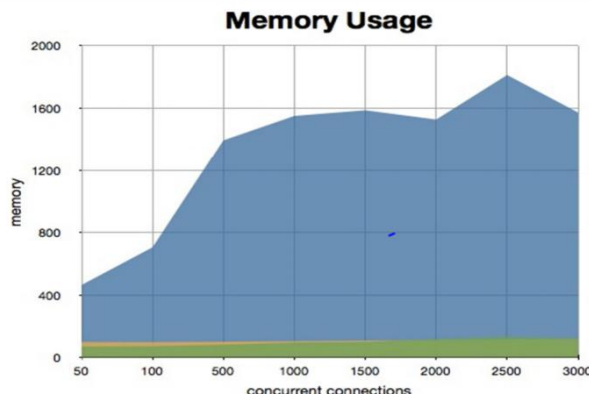


Figure. Evaluation of nginx throughput for the (a) SpecWeb, (b) Yahoo, and (c) Yahoo/2 workloads

MegaPipe (Han et al. 2012) showed a custom I/O framework increased NGINX throughput, implying NGINX has headroom for optimization

Systematic reviews [5] report NGINX usually outperforms Apache in response time, CPU, and memory usage under load.

Kim et al. (NSDI'23) [4] found NGINX and Lighttpd have very similar throughput on static content, indicating NGINX's efficiency is on par with lean servers.



Figures (from left to right). Memory usage, Requests per sec, Throughputs of lighttpd and nginx on a single CPU core

Security

- **Process isolation:** master as root only for privileged ops, workers drop to unprivileged user (limiting damage if compromised);
- **Minimal attack surface:** single-threaded workers and simple state machines reduce complexity; default config is lean (few modules);
- **Built-in mitigations:** timeouts (client_header_timeout, etc.) protect against Slowloris attacks; limit zones and buffer sizes guard against request smuggling and buffer overflow;
- **Resource limits:** OS limits (ulimits, chroot, seccomp filters if enabled) further isolate workers;
- **Reliable failover:** if a worker crashes, master immediately restarts it without dropping connections for other workers.

Current limitations



Disk I/O blocking

If a worker's disk is slow, it can still block despite non-blocking sockets; mitigated by sendfile and AIO, but still a challenge on purely single-threaded workers.



Protocol support

Native HTTP/3 (QUIC) support is only now maturing (latest NGINX versions include experimental HTTP/3) – full mainstream support is a work in progress.



Competition

Newer event-based proxies (Envoy, Traefik, OpenResty) offer features like dynamic reconfiguration, gRPC, richer plugin systems; research explores NGINX extensions (e.g. hardware offload with DPDK/DPF).

Table of references

- [1] "Alexeev. nginx. In *The Architecture of Open Source Applications*", Vol. 2, 2012. Link:
<https://aosabook.org/en/v2/nginx.html#:~:text=nginx%20runs%20several%20processes%20in,run%20as%20an%20unprivileged%20user>
- [2] "Exception-Less System Calls for Event-Driven Servers." Soares & Stumm. USENIX ATC, 2011. Link:
<https://www.eecg.toronto.edu/~stumm/Papers/Soares-USENIXATC11.pdf#:~:text=servers%20,We%20based%20our>
- [3] "Comparing the Performance of Web Server Architectures.", Pariag et al. 2007. Link:
<https://people.eecs.berkeley.edu/~brewer/cs262/Pariag07.pdf#:~:text=sockets%20that%20are%20ready%20for,blocking%20due%20to%20disk%20activity>
- [4] "Rearchitcting the TCP Stack for I/O-Offloaded Content Delivery", Taehyun Kim KAIST Deondre Martin Ng KAIST Junzhi Gong Harvard University Youngjin Kwon KAIST. Link:
<https://www.usenix.org/system/files/nsdi23-kim-taehyun.pdf#:~:text=experiments%20with%20two%20popular%20Web,refer%20to>
- [5] "Web Server Performance of Apache and Nginx: A Systematic Literature Review". Link:
https://www.researchgate.net/publication/329118749_Web_Server_Performance_of_Apache_and_Nginx_A_Systematic_Literature_Review



Thank you

Do you have any questions?

*Nginx as Unix-based
Application:*

Evolution, Architecture, Performance, Security

